OPERATING SYSTEMS PROJECT · EXPLAINED SIMPLY

Predictive Memory Paging

Teaching a computer to fetch data before it is asked for, and to know when not to

G.K Shreyas Bharadwaj (24BCE2256) · Vaastav Bajanthri (24BCE2223) · P. Giphy Bershan (24BCE2201)

Version of 19 October 2026. Every number in this document comes from the project's own result files and can be regenerated with python scripts/18_explainer_pdf.py.

The whole project in one paragraph

Computers keep the data they are using in fast memory (RAM) and the rest on a slow disk. When a program needs data that is not in RAM, it has to stop and wait: a **page fault**. We built a simulator that **predicts** which data a program will need next and loads it early, plus a **decision engine** that decides when a guess is worth the risk. Tested on 22 real programs, our final system avoids **55% of the page faults** of the standard method (LRU). But it is only faster on fast, parallel storage (SSDs), and there it is about as fast as the simple method Linux already uses.

PART 1

The problem: fast memory is small

A computer has two places to keep data. **RAM** is very fast but small and expensive. The **disk** (a hard disk or an SSD) is huge and cheap, but reading from it is thousands of times slower than reading RAM. So the operating system keeps the data a program is using right now in RAM and leaves the rest on disk.

Data moves between the two in fixed-size blocks called **pages** (4 KB each, about two pages of text). When a program asks for a page that is not in RAM, the program must stop while the page is read from disk. That pause is a **page fault**. Fewer page faults means programs wait less.

> **An everyday picture**
>
> Think of a student working at a small desk in a big library. The desk is RAM: only a few books fit, but anything on it can be read instantly. The shelves are the disk: every book is there, but walking to fetch one takes a long time. A page fault is having to stop and walk to the shelves. When the desk is full, a book must go back to the shelf to make room.

How the computer chooses which page to remove

When RAM is full and a new page must come in, some page has to leave. The rule that chooses it is the **eviction policy**. We compared six:

Policy	Rule in plain words	Library version
FIFO	Remove the page that arrived first.	Return the book that has been on the desk longest.
LRU	Remove the page used least recently. The standard choice; our main comparison.	Return the book you have not opened for the longest time.
Clock	A cheap approximation of LRU used by real operating systems.	Quickly check books in order; return the first one not touched lately.
LRU-2	Look at the second-last use instead of the last one.	Return books you have only opened once.
ARC	Balances "used recently" against "used often" and adapts.	Keep a mix of today's books and your favourite books.
OPT (Belady)	Remove the page needed furthest in the future. Impossible in reality (it needs to see the future); used as the best-possible yardstick.	A fortune-teller who knows which book you will need last.

Even OPT cannot avoid one kind of fault: the very first time a page is needed, it is not in RAM yet. Only fetching it *before* it is asked for can avoid that.

PART 2

Our idea: fetch pages before they are asked for

Classic paging only reacts: it waits for a page fault and then fetches the page. Our project makes paging **proactive**. A **predictor** watches the pages a program asks for and guesses the next ones. If a guess is right, the page is already in RAM when the program needs it, and a page fault is avoided. This is called **prefetching**.

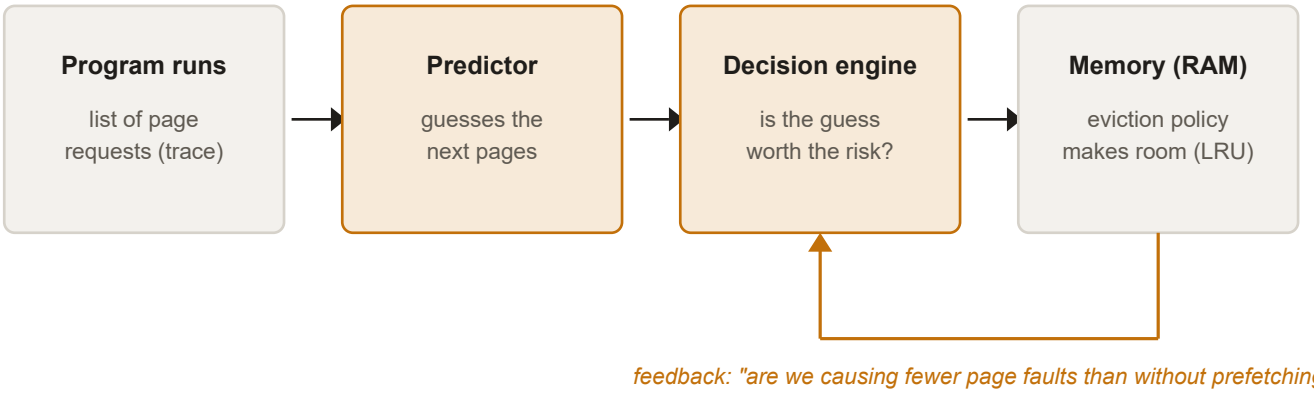
The catch: every guess is a bet

When RAM is full, loading a guessed page means throwing another page out. If the guess is wrong and the page thrown out is needed soon, prefetching has **caused** a page fault that would not otherwise have happened. A careless prefetcher can make things worse than doing nothing.

> **In the library**
>
> The student asks a helper to bring books they will probably need next. A good helper saves many trips. But the desk is small: every book the helper brings pushes another book back to the shelf. A helper who guesses badly fills the desk with useless books and the student walks to the shelves more than before.

So the system needs a second part: a **decision engine** that decides, for each guess, whether acting on it is worth the risk, and that learns from what actually happens.

How the pieces fit together



1. The **program** produces a stream of page requests. 2. The **predictor** learns from the stream and guesses what comes next. 3. The **decision engine** says yes or no to each guess. 4. Approved guesses are loaded into **RAM** through the same door as normal requests, and the **eviction policy** (LRU) decides what leaves. 5. A **feedback loop** measures whether prefetching is actually helping and makes the decision engine bolder or more careful.

One safety rule mattered throughout: prefetched pages enter RAM only through the normal path. So a bad predictor can make the system slower, but it can never make it give wrong answers.

PART 3

The predictors: how we guess the next page

We built and compared six ways of guessing. All of them are small and fast, because a predictor that runs on every memory request must cost almost nothing.

Predictor	How it guesses	Good at	Weak at
Stride	If the gaps between the last pages were equal (e.g. 10, 18, 26), guess the next one (34).	Regular steps through memory.	Anything irregular.
Markov (order 1)	Remembers "after page A, the program usually went to page B". Guesses B after A.	Repeating patterns.	Pages it has never seen.
Markov (order 2)	Same, but looks at the last *two* pages: "after A then B, usually C". More precise.	Repeating patterns, precisely.	Needs more history.
Hybrid	Tries order-2 first, then order-1, then stride.	Combines their strengths.	Still only predicts pages seen before.
Delta-Markov	Learns the *gaps* between pages instead of the pages themselves, so one rule covers a pattern anywhere in memory.	Patterns in new parts of memory.	Random access.
Readahead	The method Linux uses: when the program reads pages in order (5, 6, 7...), fetch a window of the next few pages. The window doubles while the pattern continues.	Reading through data in order.	Jumping around.

> **In everyday terms**
>
> A Markov predictor is like a friend who has watched you study for weeks: "after the chemistry book you always take the formula sheet". Readahead is simpler: "you are reading chapter 5, so bring chapters 6, 7 and 8".

Our **final predictor** combines the two that cover different situations: readahead for reading in order, and the delta-Markov predictor for repeating patterns. The Markov tables are kept small on purpose (256 entries per table; under 30 KB for all its tables together), so the predictor itself costs very little memory.

PART 4

The decision engine: when is a guess worth it?

Every guess comes with a **confidence** (how sure the predictor is). The decision engine accepts a guess only if its confidence is above a **threshold**. The interesting question is how to set that threshold. We tried nine ways:

Decision engine	How it decides	What we learned
No prefetch	Never prefetch.	The baseline: plain LRU.
Always	Act on every guess.	Works well on regular programs; can cause extra faults on irregular ones.
Fixed 0.3 / 0.6 / 0.85	Fixed confidence threshold.	Confidence turned out to be useless for this: it is never below 0.5 because of how the predictor stores data, so 0.3 behaves exactly like "always".
Precision-based	Raise the threshold when few prefetched pages get used.	Misleading. A prefetched page can be used and still have pushed out a page needed even sooner.
Fault-delta oracle	Runs a hidden second copy of memory without prefetching and compares fault counts.	Measures exactly the right thing, but a real computer cannot keep a second copy of its memory.
Ghost list	Remembers which pages a prefetch pushed out, and counts how many come back soon.	Cheap and usually right, but it can say "helping" when prefetching is actually hurting.
Shadow directory (final)	Keeps a second copy of only the page *names* (not the data), about 0.2% of RAM, and compares faults exactly.	Exact, cheap, and can be shrunk 8 times by tracking a sample of pages. This is what the final system uses.

The key question: the fault delta

The decision engine is like checking the helper's work every week: "Did I walk to the shelves less this week than I would have without the helper?" If yes, let the helper bring more books. If no, make the helper more careful. The tricky part is measuring "than I would have without the helper" without actually doing the week twice.

That measurement is called the **fault delta**: page faults without prefetching minus page faults with prefetching. Positive means prefetching helps. Finding a cheap, honest way to measure it was one of the main parts of the project.

PART 5

How we tested it

Simulation, not a real kernel. We recorded the memory requests of real programs (a **trace**) and replayed the same trace through every method. Every method sees exactly the same input, so any difference comes from the method itself. OPT also needs the recorded future, which only a replay can give.

Real programs. 22 traces from a public research benchmark set: scientific programs from SPEC CPU 2006 (fluid simulations, physics, route planning, speech recognition) and graph programs from the GAP suite (breadth-first search, PageRank, shortest paths). 200,000 memory requests each. We also used six made-up traces with known patterns (sequential, strided, looping, clustered, random, phase-changing) to check the simulator is correct.

Small memory on purpose. RAM was set to 1% and 5% of the data each program uses. With plenty of RAM nothing is ever removed and every method looks the same, so there would be nothing to test.

Fair rules.

- Every setting was chosen using only the **first quarter** of each trace, then tested once on the whole trace. Tuning on the test data would be cheating.
- Automatic checks run after every change (24 tests): no method may beat OPT, more memory must never cause more faults for LRU or OPT, and with prediction switched off the system must behave exactly like plain LRU.
- A rerun on a different computer reproduced all 48 rows of the original Google Colab results exactly.
- Results that did not work are reported too, with the reason.

What we measured

Measure	Meaning
Faults vs LRU	(LRU faults - our faults) / LRU faults. +30% means 30% fewer page faults than LRU. Negative means worse than LRU.
Speed-up vs LRU	Estimated time of LRU divided by our time. 1.5x means 1.5 times faster; below 1.0x means slower.
Prefetch precision	Share of prefetched pages that were actually used.

PART 6

The project phase by phase

The work was planned as phases. Each phase had a goal, was checked before the next one started, and sometimes changed the plan. Stages 1-5 built the system; Phases 6-13 were the original plan for improving and testing it; when results came in, the plan was revised into Phases 14-18. The table gives the overview. After it, every phase is explained in detail: **why** it was needed, **what** we did and how it works, **how we checked** it, the **result**, and the **lesson** it taught us.

Phase	Goal	Status	Outcome in one line
Stage 1	Get the data	done	6 test traces + real benchmark traces converted to pages
Stage 2	Prepare the data	done	One common format; first 25% kept aside for choosing settings
Stage 3	Build the predictor	done	Hybrid Markov predictor; right about the next page 81% of the time
Stage 4	Build the memory engine	done	FIFO, LRU, Clock, OPT with automatic correctness checks
Stage 5	Build the decision engine	done	Fault-delta feedback halves the worst damage
Stage 5b	Compare with classic methods	done	35-37% fewer faults on two regular programs
Stage 5c	Make the feedback deployable	done	Ghost list matches the impossible "oracle"
Phase 6	Testing harness	built, replaced by Phase 17	Built early; the final large test (Phase 17) replaced it
Phase 7	Clean up and verify	done	Local rerun matches the original results exactly (48 of 48)
Phase 8	Stronger baselines	done	Surprise: Linux readahead beats our Markov predictor on 3 of 4
Phase 9	Smarter eviction	done: negative result	None of the eviction ideas helped; combining with readahead did
Phase 10	Better predictor	done	Delta-Markov added; top-2 mixed; smoothing rejected
Phase 12a	First robustness run	done	13 programs: combined system 63.5% fewer faults on average
Phase 12b	Cost of extra reads	done	No system wins if a prefetch costs as much as a fault
Phase 12c	Let the decider stop readahead	done: partly worked	Worst case halved; costs some gain on mcf
Phase 14	Fix the feedback signal	done	Shadow directory: a bad case from -21.5% to -2.5%
Phase 15	Time model	done	Faster on SSD, slower on hard disk; timing matters
Phase 16	Tune and freeze settings	done	Faster-reacting controller, chosen on warm-up data only
Phase 17	Test on 22 programs	done	55% fewer faults; a hidden 35% cap found and removed

Phase	Goal	Status	Outcome in one line
Phase 18	Write-up	draft done	Report, paper draft, component comparison, this document

Phases 11 (tuning), 12 (robustness) and 13 (write-up) of the original plan were carried out as Phases 16, 17 and 18.

Building the system (Stages 1-5)

STAGE 1 Getting the data — done

Why	To test an idea about memory you need recordings of how real programs use memory, plus simple recordings where you already know the right answer, so you can tell whether the simulator itself is correct.
What we did	Wrote six generators for made-up patterns: *sequential* (1, 2, 3, 4, ...), *strided* (0, 8, 16, 24, ...), *cyclic* (a loop of 20 pages, repeated), *clustered* (random jumps inside a group of 10 pages, then a move to a new group every 500 requests), *random*, and *phase-changing* (sequential, then cyclic, then random).
Real programs	Downloaded recordings from a public research competition (ML-DPC LoadTraces). They cover scientific programs from SPEC CPU 2006 (bwaves and leslie3d simulate fluids) and graph programs from the GAP suite (bfs = breadth-first search, pr = PageRank). Each line of a recording holds a memory address; dividing the address by 4,096 (the size of one 4 KB page) turns it into a page number. Example: address 0x7f3a2c18 becomes page 0x7f3a2.
How we checked	A broken download (the server sometimes returns a web page instead of data) is detected and rejected automatically. The made-up patterns act as tests: for example, a sequential scan with no prediction must cause a fault on every single request.
Result	Four real programs, 200,000 requests each, using between 2,134 and 5,846 distinct pages (bwaves 3,051; leslie3d 2,134; bfs-3 5,846; pr-3 4,062).
Lesson / caveat	The recordings come from the processor's cache, not from the operating system. Treating them as pages is a close approximation of real paging, not the real thing. Always say this.

STAGE 2 Preparing the data — done

Why	Every method must see exactly the same input, and choosing settings must never use the data the methods are judged on.
What we did	Every trace, real or made-up, became the same simple object: an ordered list of page numbers plus the gaps between them. The first 25% of each trace, in time order, was set aside as the **warm-up part**. The order is never shuffled, because the order of requests is exactly what the predictor tries to learn.
How we checked	Automatic checks that every trace loads as the right kind of object. A real bug was caught this way: one notebook cell overwrote the traces with plain lists.
Lesson	The rule "settings may only be chosen on the warm-up part" runs through the whole project. Tuning on the test data would be cheating, because the results would then look better than they really are.

STAGE 3 The predictor — done

Why	Prefetching needs a guess about which page comes next, and the guess must be cheap, because it is made on every single memory request.
What we did	Built four predictors. **Stride**: if the last gaps were equal (10, 18, 26: gap 8), guess the next one (34). **Markov order 1**: a table that counts "after page A, the next page was B"; when A appears again, it guesses the B seen most often. **Markov order 2**: the same, but remembers the last *two* pages ("after A then B came C"), which is more precise. **Hybrid**: asks order 2 first, then order 1, then stride, and uses the first answer.
How it works	The tables are deliberately small: 256 entries, only the two most frequent next pages kept per entry, whole-number counts, and every count halved every 25,000 updates so old habits fade. Confidence = how often the top next page was seen, divided by all counts in that entry.

How we checked	Measured how often the top guess is exactly the next page, which sub-predictor answered, and how much memory the tables use. Settings were chosen on warm-up data: accuracy was identical from 128 to 4,096 entries, so 256 was enough.
Result	The hybrid's top guess was exactly the next page 81% of the time on the four real programs. Its tables take a few KB.
Lesson	Markov can only predict pages it has already seen, so a program reading brand-new data in order defeats it. And because only two next pages are kept, the top one always holds at least half the counts, so confidence is never below 0.5. Both facts mattered later.

STAGE 4 The memory engine

done

Why	We need a correct simulation of RAM and of the classic ways of choosing which page to remove.
What we did	Built **FIFO** (remove the oldest page), **LRU** (remove the least recently used page: the standard, and our main comparison), **Clock** (a cheap version of LRU used by real systems) and **OPT** (remove the page needed furthest in the future). OPT is impossible in a real computer but possible in a simulator, because the recording shows the future; it is the best any eviction policy could do.
How it works	Every request goes through one "install" path: if the page is missing it is a fault, the policy picks a victim if RAM is full, and the page is loaded. Prefetched pages later use the same path, so a bad predictor can slow the system down but can never make it give wrong answers. RAM is sized as a percentage of each program's data (1%, 2%, 5%, ...), because with a fixed large memory nothing is ever removed and every policy gives identical results.
How we checked	Automatic checks: no policy may beat OPT; more memory must never mean more faults for LRU and OPT; a sequential scan must fault on every request. All passed.
Result	leslie3d: LRU 15,313 faults vs OPT 5,881, so 62% room for improvement. bwaves: almost none (about 1%). On some settings LRU was even worse than FIFO.
Lesson	OPT still pays for the first visit to every page. Only fetching pages early (prefetching) can avoid those first-visit faults.

STAGE 5 The decision engine

done

Why	When RAM is full, every prefetch removes a page. A wrong guess can push out a page needed soon and create a fault. We need a rule for when acting on a guess is worth it.
What we did	Compared seven engines, all given exactly the same guesses, on eight test settings: **never** (plain LRU); **always**; **fixed thresholds** (act only if confidence is at least 0.3, 0.6 or 0.85); **precision-based** (become more careful when few prefetched pages are used); and **fault delta**.
How fault delta works	A hidden second copy of memory runs the same requests *without* prefetching. Every 2,000 requests the engine compares: if the real system had fewer faults than the copy, prefetching is helping, so it lowers its threshold (bolder); if more, it raises it (more careful); if about equal, it leaves it.
Result	**Confidence was useless**: it is never below 0.5, so "0.3" behaved exactly like "always". **Precision misled**: on the clustered test 81-86% of prefetched pages were used and faults still went up, because a used prefetch can still push out a page needed even sooner. **Fault delta was best**: it halved the worst damage (-0.72% vs -1.50% for "always") and kept 99.94% of the benefit on leslie3d.
Lesson	The right signal is the outcome itself: "are we causing fewer faults than without prefetching?" Indirect signals (confidence, precision) can point the wrong way.

STAGE 5b Head-to-head with the classic methods — done

What we did	Added prediction with the fault-delta engine on top of FIFO, Clock and LRU, and compared with all four classic methods.
Result	leslie3d: 15,313 to 9,956 faults (35% fewer). bwaves: 3,140 to 1,969 (37% fewer), even below OPT's 3,112, because prefetching avoids first visits. The gain was about the same on every policy (34.5-38.6%), so prediction adds to any eviction policy. The graph programs gained almost nothing: over 99% of their guesses named a page that was already in RAM.

STAGE 5c The ghost list — done

Why	The fault-delta engine needs a hidden second copy of memory. A real computer cannot afford to keep its memory twice.
What we did	A short list (256 entries) that remembers only the *names* of pages a prefetch pushed out, and when. If one of them is needed again within 2,000 requests, that counts as harm. Estimated benefit = prefetched pages that were used - harm.
Result	As good as the hidden-copy version: average gain 9.02% vs 8.98%. Caveat: on pr-3 the estimate moved opposite to the truth (correlation -0.98), although it never caused a wrong decision there.

Checking, comparing and improving (Phases 6-10)

PHASE 6 Testing harness — built, replaced by Phase 17

What we did	Built a script to test other recording segments, several memory sizes and several random seeds, and to draw the figures.
What happened	It was deferred so it could run on the final system rather than an early one. The 22-program test of Phase 17 covers the same ground.

PHASE 7 Clean-up and verification — done

Why	The early work lived in a Google Colab notebook. Before building further, everything had to be reproducible and checked.
What we did	Turned the notebook into a project with one script per step and automatic tests. Re-downloaded the traces on the laptop and reran everything from scratch.
How we checked	All 48 rows of the original decision-engine results came out **identical**, so the traces and the simulator are exactly the same as before. An independent, straight-from-the-definition version of LRU-2 matched ours fault for fault.
Problems found	A hidden trap: passing the ghost-list engine to the wrong function silently turned it into the hidden-copy version; it is now blocked. The claim "confidence is never below 0.5" turned out to come from keeping only two next pages, not from the programs; the documents were corrected.

PHASE 8 Stronger baselines — done

Why	A reviewer would ask: why compare only with textbook methods? We added stronger ones.
What we did	Added **ARC** (balances recently used against frequently used pages and adapts), **LRU-2** (looks at the second-last use instead of the last) and **Linux-style readahead**: when pages are read in order (5, 6, 7, ...), fetch a window of the next few pages; the window doubles while the run continues and resets when it breaks.

Result	ARC and LRU-2 alone did not help (ARC: 16,553 faults vs LRU 15,313 on leslie3d). Prediction still added 34-36% on top of ARC. **Surprise: readahead beat our Markov predictor on 3 of 4 programs** (bwaves 204 vs 1,969 faults; bfs-3 2,924 vs 7,427; pr-3 2,905 vs 5,872).
Lesson	Readahead fetches *new* pages in a run, which a Markov predictor can never predict because it has never seen them. This changed the direction of the project: the two had to be combined.

PHASE 9 Smarter eviction

done: negative result

Why	So far the predictor only decided what comes in. The plan was to let it also influence what goes out.
What we tried	(1) Remove prefetched pages that were not used after a while, before any normal page. (2) Put prefetched pages last in line, so they leave first unless used. (3) Never remove the page the predictor expects next. All tested on warm-up data first.
Result	None beat plain LRU. Protecting prefetched pages for longer was much worse, because it pushed out pages the program really needed. Protecting the expected page changed nothing, because that page is almost always already in RAM and recently used, so LRU keeps it anyway.
What we did instead	Combined readahead and Markov into one predictor: readahead's window first, then the Markov guess. On leslie3d: 8,308 faults, better than Markov (9,956) or readahead (11,624) alone.
Lesson	A negative result with a reason is still a result: the gains come from what is fetched, not from cleverer removal.

PHASE 10 Better predictor

done

Delta-Markov	A Markov table over the *gaps* between pages instead of the pages themselves. A rule like "gap +1, then +3" then works anywhere in memory, even on pages never seen before. Markov alone on bwaves: 1,969 to 810 faults; combined system: leslie3d 8,131 and bfs-3 2,570 (now better than readahead's 2,924). **Adopted.**
Top-2	Prefetch the two best guesses instead of one. Big win on leslie3d (5,287 faults, below OPT) but 20% worse on bfs-3 and slightly worse on bwaves. **Mixed: not adopted.**
Other ideas	Smoothed confidence: slightly worse. Requiring a longer run before readahead starts: fixed the clustered test but lost the graph gains. **Both rejected.**
How we checked	Every choice made on the warm-up part first, then run once on the full traces.

Testing honestly and finishing (Phases 12a-18)

PHASE 12a First robustness run (13 programs)

done

Why	Four programs are not enough to trust a result. It might be luck.
What we did	Downloaded 9 more recordings (two more segments each of bwaves and leslie3d, second inputs for bfs and pr, and milc, libquantum, mcf). Ran every system at 1% and 5% memory with no retuning: 26 tests.
Result	Combined system: 63.5% fewer faults than LRU on average, better than LRU in 25 of 26 tests. Readahead alone: 57.4%. Original Markov: 10.1%. The one loss was bwaves-s2, which has only 8 frames of memory.

PHASE 12b Cost of the extra disk reads — done

Why	Fault counts hide a cost: every prefetch is also a disk read.
What we did	For each system we computed the **break-even cost**: how expensive one prefetch may be, as a fraction of one fault, before the system loses to LRU. Formula: (LRU faults - our faults) / number of prefetches.
Result	Every value was below 1: if a prefetch costs as much as a fault, every system loses. The original Markov system used its reads most efficiently (it wins on leslie3d as long as a prefetch costs less than about 0.9 of a fault); the combined system needs prefetches to be much cheaper (below about 0.4 on leslie3d).

PHASE 12c Letting the decision engine stop readahead — done: partly worked

Why	Readahead's guesses always had confidence 1.0, so the decision engine could never block them, and on the clustered test random hops looked like runs.
What we did	Gave readahead's guesses a confidence that grows with the length of the run: run / (run + 1). A run of 1 gives 0.5, a run of 9 gives 0.9. The setting was chosen on warm-up data.
Result	The worst case was halved (-6.3% to -3.2%), but the gain on mcf dropped (10.7% to 5.8%) and the clustered test with 9 frames did not improve at all.
Diagnosis	On that test the ghost list said "winning" in 24 of 25 periods while prefetching was really losing. It counts every used prefetch as a saved fault, the same mistake precision made in Stage 5. This led to Phase 14.

PHASE 14 Fixing the feedback signal — done

Why	The ghost list could be fooled, and the exact version needed a full second copy of memory.
What we did	Built the **shadow directory**: a second copy of memory that holds only page *names*, never the data, and runs without prefetching. It costs 8 bytes per page frame, about 0.2% of RAM. Comparing its faults with the real system's gives the exact fault delta. To make it even cheaper, it can track only a sample of pages chosen by a hash (an idea from the SHARDS paper) and scale up the count.
Also tried	An "off switch" for when prefetching keeps losing, with retries after 1, 2, 4, ... periods. It never triggered on any program, so it stays off.
How we checked	With every page tracked, the estimate equalled the true value in every single period (a pinned automatic test). Tracking only 1 page in 8 gave the same fault counts on 9 of 10 programs.
Result	The clustered test went from 21.5% worse than LRU to 2.5% worse. Over 26 real tests it had the best worst case of every system (-1.4%), and it recovered the mcf gain lost in Phase 12c.

PHASE 15 Time model — done

Why	Fewer page faults are only useful if the program finishes sooner. Fault counts ignore late prefetches, extra disk reads and queues.
What we did	Replayed every run on a simulated disk. Between requests the program computes for a while (unknown for these recordings, so we tried 0.1, 1 and 10 microseconds). Each read takes 80 microseconds on a fast SSD that can serve 8 reads at once, or 5 milliseconds on a hard disk that serves one at a time. A fault makes the program wait; a prefetch does not, but if its page is needed before the read finishes, the program waits for the rest. Each prediction costs 0.2 microseconds of CPU.
How we checked	Automatic tests of the model itself: with no prefetching the time is exactly computing time plus fault time; late prefetches and queueing are charged correctly.

Result	Fast SSD: the combined system was about 1.4-1.5 times faster than LRU. Hard disk: every prefetcher was slower than LRU. The original Markov system avoided 35-37% of faults on leslie3d and bwaves yet was not faster: it fetches only one page ahead, so its pages often arrive late (16-22% of the run time was spent waiting for late prefetches).
Lesson	Being right is not enough: a prefetch must arrive in time, and the disk must be able to do several reads at once.

PHASE 16 Tune and freeze the settings

done

Why	The decision engine's settings (how often it reacts, how big each step is, how big the "no change" zone is) had never been tuned.
What we did	Tried 36 combinations on the warm-up part of 14 programs only. The rule for picking the winner was written down before the run: keep settings whose worst case is no worse than the old one, take the best average, and keep the old setting unless the new one is clearly better.
Result	Chosen: react every 1,000 requests (was 2,000) in steps of 0.1 (was 0.05). The warm-up worst case improved from -8.0% to -3.8%. Only the two programs where prefetching can hurt changed; the other twelve were identical. The settings were frozen; the final system is fixed from here on.

PHASE 17 The final test: 22 real programs

done

Why	The real test of a frozen system is programs it has never been tuned on.
What we did	Added 9 more programs (soplex, omnetpp, astar, GemsFDTD, lbm, sphinx3 from SPEC; bc, cc, sssp from GAP) and ran the frozen final system at 1% and 5% memory: 44 tests.
Found on the way	A 35% cap on prefetches, left over from Stage 5, stopped the system on omnetpp at exactly 35% of requests while it was winning every single period. Choosing on warm-up data only, the cap was removed (omnetpp warm-up: 56% to 97% of faults avoided; no program got worse).
Result	55.0% fewer faults than LRU on average over 44 tests; worse than LRU only once (-3.3%, bwaves-s2); fewer faults than readahead in 34 of 44. In time on the SSD model: 1.43x vs readahead's 1.44x, a tie. Hard disk: slower than LRU.
Lesson	The final system avoids the most faults of everything we tested, but in time it only matches the simple method Linux already uses.

PHASE 18 Write-up

draft done

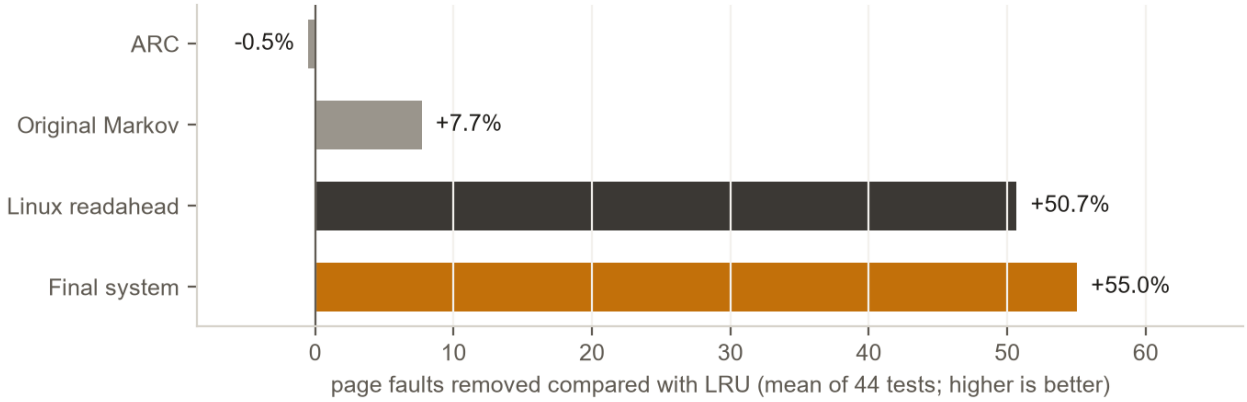
What we did	An interactive report of every result (results/report.html); a research paper draft (docs/paper_draft.md); a side-by-side comparison of all 6 predictors and 9 decision engines (Part 8 of this document); and this document.
Still to do	Full citations for the reference papers, choosing figures for the paper, and a new slide deck from the team's screenshots.

PART 7

The results

Result 1: the final system avoids the most page faults

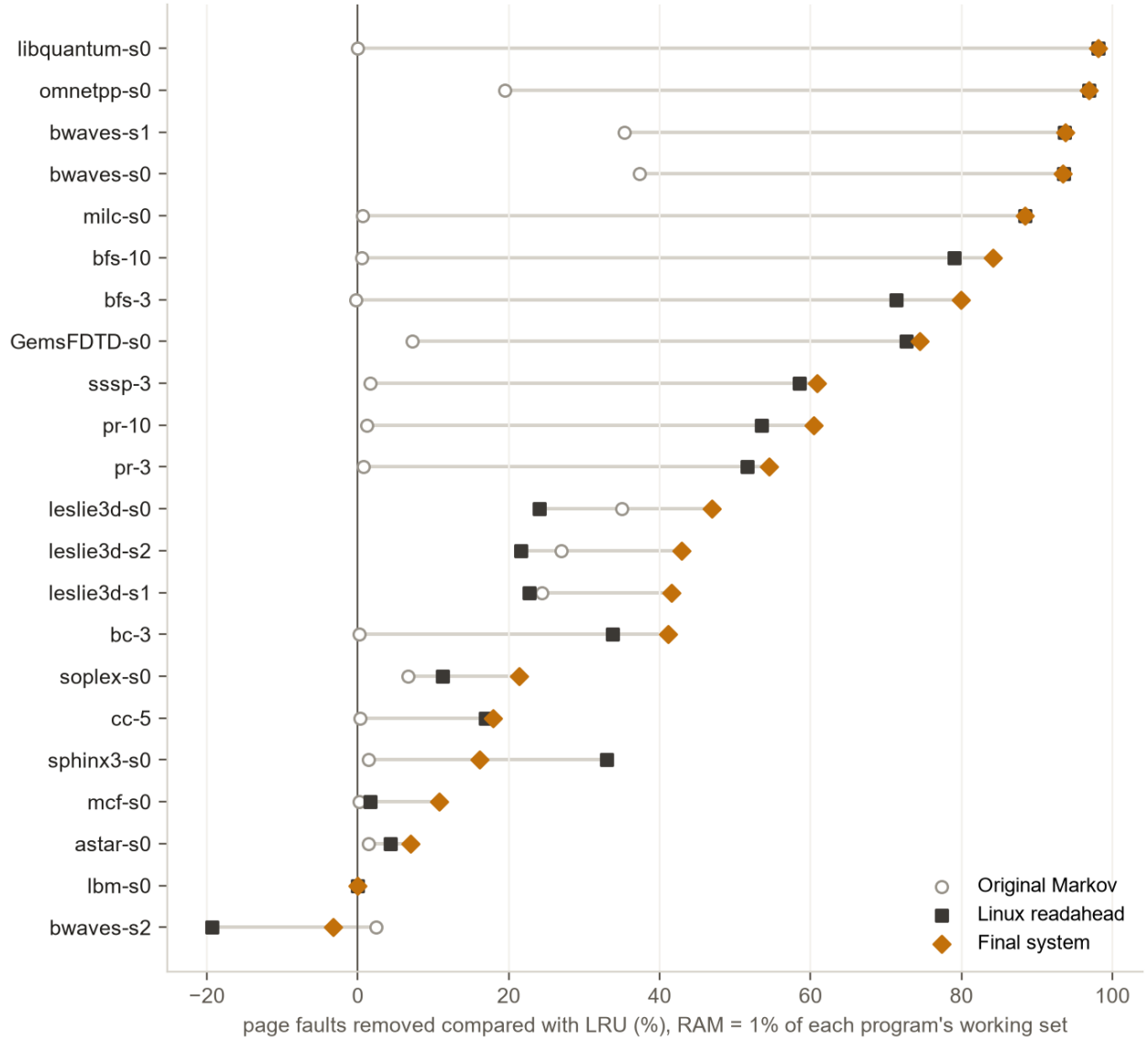
Averaged over 44 tests (22 real programs, each with two memory sizes), compared with LRU:



Mean share of LRU's page faults that each system avoids. ARC is a smarter eviction policy without prediction; it does not help on these programs.

System	Faults avoided (mean)	Worst case	Worse than LRU in
ARC (smarter eviction, no prediction)	-0.5%	-8.2%	22 of 44
Original Markov system	+7.7%	-0.3%	1 of 44
Linux readahead	+50.7%	-19.3%	2 of 44
Final system	**+55.0%**	**-3.3%**	**1 of 44**

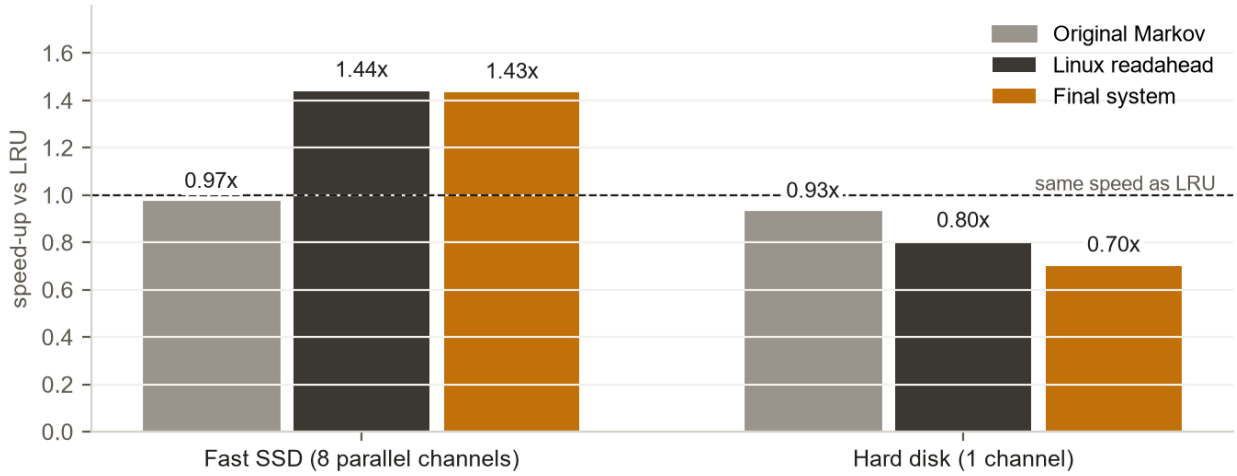
The final system has fewer faults than Linux readahead in 34 of 44 tests, the same in 8 and more in 2. In 30 tests it even beats OPT, the "fortune-teller" policy: OPT is the best possible way to *remove* pages, but it still pays for every first visit, which prefetching can avoid.



Each row is one real program (RAM = 1% of its data). Further right is better. The original Markov system helps on only a few programs; readahead and the final system help on most. lbm has no pattern anyone can predict.

Result 2: fewer page faults does not always mean faster

Fault counts hide a cost: every prefetch is a disk read too. We built a time model that replays each run on a simulated disk, including the time the program computes between requests, reads that queue behind each other, and prefetches that arrive too late. Two disks were modelled: a fast SSD that can serve 8 reads at once, and a hard disk that serves one at a time.



Average speed-up compared with LRU over 22 programs (1 microsecond of computing between requests, prediction cost included). Above the dashed line = faster than LRU.

System	Fast SSD	Hard disk
Original Markov	0.97x (slower on 18 of 22)	0.93x
Linux readahead	1.44x (slower on 2 of 22)	0.80x
Final system	**1.43x** (slower on 3 of 22)	**0.70x**

- **On a fast SSD** the final system is about 1.4 times faster than LRU, but so is Linux readahead: the two are level. The extra faults our system avoids are paid for with extra disk reads.
- **On a hard disk every prefetcher is slower than LRU.** A hard disk does one read at a time, so each prefetch costs as much as the fault it tries to save.
- **Timing matters as much as accuracy.** The original Markov system avoids 35% of the faults on leslie3d and 37% on bwaves, yet it is not faster: it fetches only one page ahead, so the page often arrives after the program already needs it. Readahead fetches several pages ahead and arrives in time.

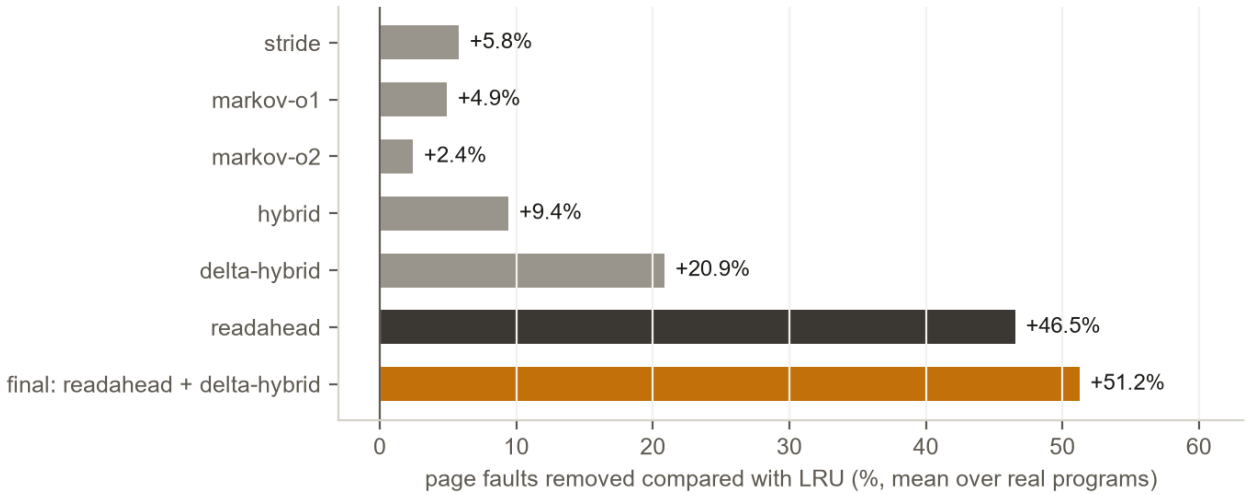
> **Why "right" is not enough**
>
> A helper who brings the right book just after you stood up to fetch it yourself saves you nothing. A helper who brings the next three chapters while you are still reading saves real time, but only if the library has several helpers who can walk at once (an SSD). With a single slow helper (a hard disk), every extra trip delays the books you actually asked for.

PART 8

Every component side by side

To see what each piece contributes, we changed one component at a time and kept everything else the same, on every real program at 1% memory with LRU underneath.

The six predictors (same decision engine for all)



Predictor	Correct next-page guesses	Faults avoided	Worse than LRU	Speed-up (SSD)
stride	3%	+5.8%	0	1.01x
markov-o1	50%	+4.9%	3	1.01x
markov-o2	53%	+2.4%	3	1.00x
hybrid	57%	+9.4%	1	1.01x
delta-hybrid	59%	+20.9%	0	1.02x
readahead	3%	+46.5%	1	1.51x
final (readahead + delta-hybrid)	56%	+51.2%	1	1.49x

"Correct next-page guesses" is how often the top guess is exactly the next page. It does **not** predict who wins on page faults: readahead guesses the very next page less often than Markov, but it avoids far more faults because it fetches pages the program has never touched before.

The nine decision engines (all fed the final predictor's guesses)

Decision engine	Faults avoided (mean, real programs)	Worst real program	Danger test (clustered)	Speed-up (SSD)
no-prefetch	+0.0%	+0.0%	+0.0%	1.00x
always	+51.2%	-6.3%	-21.5%	1.49x
fixed-0.3	+51.2%	-6.3%	-21.5%	1.49x
fixed-0.6	+28.8%	+0.0%	-5.5%	1.21x
fixed-0.85	+19.8%	+0.0%	-0.2%	1.07x
precision (cost-aware)	+26.8%	+0.0%	-5.2%	1.16x
fault-delta oracle	+49.1%	+0.0%	-1.7%	1.39x
ghost list	+47.9%	-3.2%	-21.5%	1.39x
shadow directory	+49.0%	-1.4%	-2.5%	1.39x
directory + frozen controller (final)	+51.2%	-3.3%	-2.4%	1.49x

"Danger test" is the made-up clustered pattern, where prefetching tends to cause harm. Reading the table: **always** gets the full benefit on real programs but is badly hurt on the danger test. **Fixed thresholds** are safe but throw away half or more of the benefit. The **oracle** is never worse than LRU but needs a second copy of memory, which a real computer cannot keep. The **final engine** (shadow directory with tuned settings) keeps the full benefit and limits the danger-test damage to a few percent. With the original Markov predictor (not shown) every engine gives about the same result, because that predictor rarely has anything worth prefetching.

PART 9

What did not work, and why

Ideas that failed are part of the result: they show where the gains really come from.

Idea	What happened	Why
Remove unused prefetched pages early	Never better than LRU; worse when prefetched pages were kept longer.	Protecting guesses pushes out pages the program really needs.
Never remove the page the predictor expects next	No change at all.	That page is almost always already in RAM and recently used, so LRU keeps it anyway.
Start readahead only after longer runs	Fixed one test pattern but lost the gains on graph programs.	Graph programs read many short runs.
Smooth the confidence numbers	Slightly worse.	The decision engine already reacts to the real outcome.
Using precision as the feedback signal	Sometimes the worst result of all.	A used prefetch is not always a helpful one.
A 35% cap on prefetches (from the first design)	Blocked the system on omnetpp while it was clearly winning.	A fixed safety limit is not needed once the feedback is exact. Removed.

What to be careful about (limitations)

- The traces come from the processor's cache, treated as pages. That is close to, but not the same as, real operating-system paging.
- The time results come from a model with typical disk speeds, not from measurements on a real machine.
- Each program was run once (the traces are fixed, so reruns give identical numbers); 200,000 requests per program; small simulated memories.
- In our simulator the predictor sees every memory request. A real operating system only sees page faults, so in practice it would know less.
- The building blocks are known ideas (Markov prefetching from 1997, Linux readahead, ghost lists from ARC and Linux). Our contribution is how they are combined and, above all, the careful evaluation.

PART 10

How to explain this project to someone else

In 30 seconds

> "When a program needs data that is not in fast memory, it has to wait for the disk. We built a simulator that predicts the next data and loads it early, plus a decision engine that only does so when it helps. On 22 real programs it avoids about 55% of the waits compared with the standard method. The twist: on a fast SSD it is about 1.4x faster, but no faster than what Linux already does, and on a hard disk prefetching makes things slower."

In 3 minutes: the story arc

- **Problem:** RAM is small, disks are slow, page faults make programs wait.
- **Idea:** predict and prefetch, but every wrong guess can push out a useful page, so you need a decision engine.
- **Lesson 1:** the right signal for the decision engine is "are we causing fewer faults than without prefetching?" Confidence and precision both mislead. We found a cheap, exact way to measure it (the shadow directory).
- **Lesson 2:** combining a pattern-learning predictor with simple readahead avoids the most faults.
- **Lesson 3:** fewer faults is not the same as faster. Timing, extra disk reads and the type of disk decide whether prefetching pays off.

Questions people often ask

Question	Short answer
Did you build a new algorithm?	No. The pieces are known. What we contribute is how they are combined and a careful, honest test of which feedback signal works and when prefetching actually saves time.
Why not use AI / deep learning?	The predictor runs on every memory request, so it must be tiny and fast. A learned model is a fair comparison for future work.
How can anything beat OPT?	OPT is the best at choosing what to remove, but it still waits for every first visit. Prefetching avoids first visits.
How do you know the simulator is right?	Automatic checks after every change, test traces with known answers, and an exact rerun of the original results on another computer.
So is it useful?	It avoids the most page faults of everything we tested. Whether that saves time depends on the disk: yes on fast SSDs (but no more than Linux readahead), no on hard disks.

PART 11

Glossary

Term	Meaning
RAM	Fast, small working memory.
Disk (HDD / SSD)	Large, slow storage. A hard disk (HDD) reads one thing at a time; a modern SSD can serve several reads at once.
Page	A fixed-size block of data (4 KB) moved between disk and RAM.
Page fault	The program asks for a page that is not in RAM and must wait for the disk.
Eviction policy	The rule that decides which page leaves RAM when space is needed (FIFO, LRU, Clock, ARC, LRU-2, OPT).
Prefetching	Loading a page before it is asked for.
Predictor	The part that guesses the next pages (Markov, readahead, ...).
Markov predictor	Remembers which page usually follows which, and guesses accordingly.
Readahead	When pages are read in order, fetch the next few pages too (what Linux does).
Decision engine	Decides whether a guess is worth acting on.
Fault delta	Faults without prefetching minus faults with it. Positive = prefetching helps.
Ghost list / shadow directory	Cheap ways to estimate the fault delta: remember page names, not page data.
Trace	A recording of the page requests a real program made.
Working set	All the distinct pages a program uses.
Warm-up portion	The first 25% of each trace, the only part used to choose settings.

How to run the project

From the project folder: python -m pip install -r requirements.txt -r requirements-dev.txt, then python -m pytest -q (should print 24 passed), then python scripts/run_all.py (15-25 minutes) and open results/report.html for every table and chart. If the terminal says "python: command not found", use py instead of python.

Where to read more: docs/RESULTS.md (every number with its source), docs/paper_draft.md (the research paper draft), results/report.html (all results, interactive).